SOFTWARE ARCHITECTURE

MVC Architecture

BEYOND BOUNDARIES

HOA SEN UNIVERSITY
WORLD CLASS EDUCATION

QS STARS RATING SYSTEM

ACBSP ACCREDITED

AUN-QA

MOET ACCREDITED
KIỂM ĐỊNH CHẤT LƯỢNG GIÁO DỤC QUỐC GIA

Content

- History
- MVC Pattern
- Building Loosely Couple Components

The History of MVC

- The term model-view-controller has been in use since the late 1970s and arose from the Smalltalk project at Xerox PARC, where it was conceived as a way to organize some early GUI applications.
- Interactions with an MVC application follow a natural cycle of user actions and view updates, where the view is assumed to be stateless. This fits nicely with the HTTP requests and responses that underpin a Web application.
- Further, MVC forces a separation of concerns—the domain model and controller logic are decoupled from the user interface.
- MVC and it remains the implementation template for the MVC pattern.
- Many other MVC frameworks have since emerged and demonstrated the benefits of MVC—including, of course, ASP.NET MVC.

Understanding the MVC Pattern (1/2)

- **Models**, which contain or represent the data that users work with.
- **Views**, which are used to render some part of the model as a user interface.
- **Controllers**, which process incoming requests, perform operations on the model, and select views to render to the user.

Understanding the MVC Pattern (2/2)

- **Models** are the definition of the universe your application works in. In a banking application, for example, the model represents everything in the bank that the application supports, such as accounts, the general ledger, and credit limits for customers
- **Views** contain the logic required to display elements of the model to the user—and nothing more. They have no direct awareness of the model and do not directly communicate with the model in any way.
- **Controllers** are the bridge between views and the model—requests come in from the client and are serviced by the controller, which selects an appropriate view to show the user and, if required, an appropriate operation to perform on the model.

The Domain Model

- The most important part of an MVC application is the domain model.
- We create the model by identifying the real-world entities, operations, and rules that exist in the industry or activity that the application must support, known as the **domain**.
- We then create a software representation of the domain: the **domain model.**
- When an instance of a domain type is created to represent a specific piece of data, it is called a **domain object**.
- Domain models are usually persistent and long-lived—there are lots of different ways of achieving this, but relational databases remain the most common choice.

The Domain Model

- In short, a domain model is the single, authoritative definition of the business data and processes within your application.
- A persistent domain model is also the authoritative definition of the state of your domain representation.
- The domain model approach solves many of the problems that arise when maintaining an application.
- If you need to manipulate the data in your model or add a new process or rule, the domain model is the only part of your application that must be changed.

Comparing MVC to Other Patterns

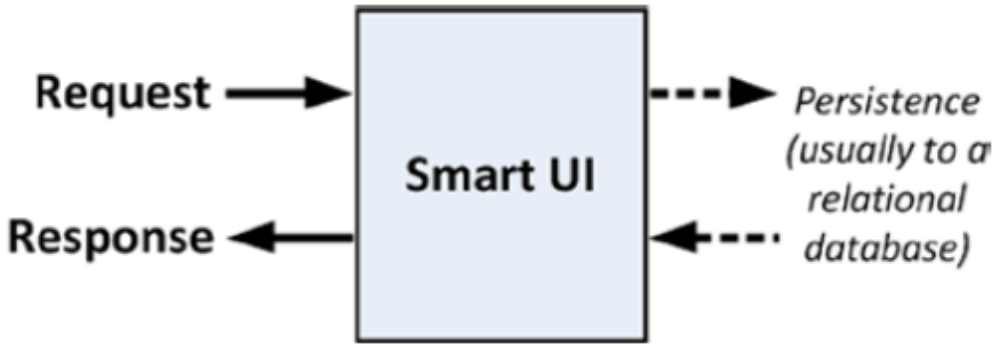
- MVC is not the only software architecture pattern. There are many others and some of them are, or at least have been, extremely popular.
- We can learn a lot about MVC by looking at the alternatives as:
  - The Smart UI Pattern
  - The Model-View Architecture
  - The Classic Three-Tier Architectures

The Smart UI Pattern (1/2)

- To build a smart UI application, developers construct a user interface, often by dragging a set of components or controls onto a design surface or canvas.
- The controls report interactions with the user by emitting events for button presses, keystrokes, mouse movements, and so on.
- The developer adds code to respond to these events in a series of event handlers: small blocks of code that are called when a specific event on a specific component is emitted.

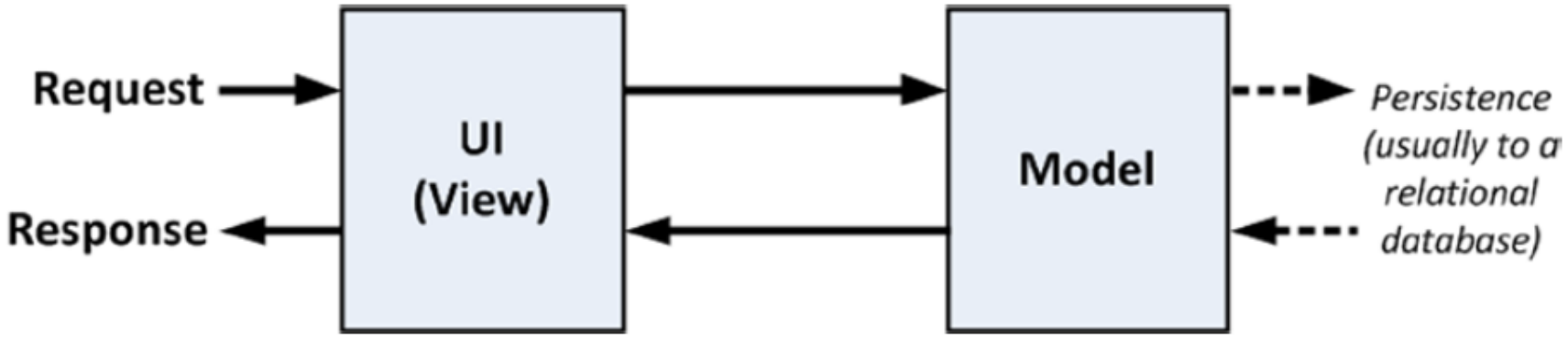
The Smart UI Pattern (2/2)

- This creates a monolithic application. The code that handles the user interface and the business is all mixed together with no separation of concerns at all.
- The code that defines the acceptable values for a data input, that queries for data or modifies a user account, ends up in little pieces, coupled together by the order in which events are expected.
- The Smart UI is often referred to as an anti-pattern



The Model-View Architecture (1/2)

- Pulls out the business logic into a separate domain model.
- The data, processes, and rules are all concentrated in one part of the application

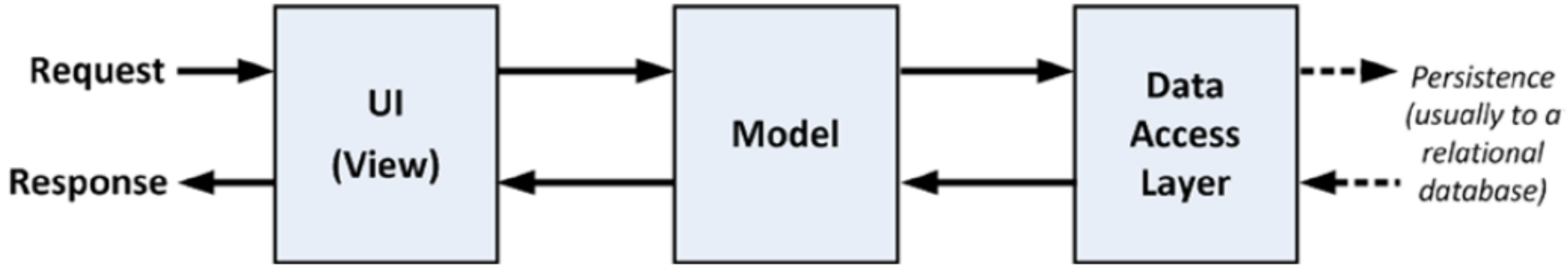


The Model-View Architecture (2/2)

- Two problems
  - Since the UI and the domain model are closely integrated, it can be difficult to perform unit testing on either.
  - From practice, rather than the definition of the pattern. The model typically contains a mass of data access code—this need not be the case, but it usually is—and this means that the data model does not contain just the business data, operations, and rules.

The Classic Three-Tier Architectures

- The three-tier architecture is the most widely used pattern for business applications.
- The DAL can be created so that unit testing is relatively easy. You can see the obvious similarities between a classic three-tier application and the MVC pattern.
- The difference is that when the UI layer is directly coupled to a click-and-event GUI framework, it becomes almost impossible to perform automated unit tests.



Variations on MVC

- The Model-View-Presenter Pattern
- The Model-View-View Model Pattern

The Model-View-Presenter Pattern (1/2)

- Model-view-presenter (MVP) is a variation on MVC that is designed to fit more easily with stateful GUI platforms such as Windows Forms or ASP.NET Web Forms.
- This is a worthwhile attempt to get the best aspects of the Smart UI pattern without the problems it usually brings

The Model-View-Presenter Pattern (2/2)

- There are two implementations of this pattern:
  - The passive view implementation, in which the view contains no logic—it is a container for UI controls that are directly manipulated by the presenter.
  - The supervising controller implementation, in which the view may be responsible for some elements of presentation logic, such as data binding, and has been given a reference to a data source from the domain models.
- The difference between these two approaches relates to how intelligent the view is.
- Either way, the presenter is decoupled from the GUI framework, which makes the presenter logic simpler and suitable for unit testing.

The Model-View-View Model Pattern

- The model-view-view model (MVVM) pattern is the most recent variation on MVC.
- It originated from Microsoft and is used in the Windows Presentation Foundation (WPF).
- In the MVVM pattern, models and views have the same roles as they do in MVC.
- The difference is the MVVM concept of a view model, which is an abstract representation of a user interface
- Unlike an MVC controller, an MVVM view model has no notion that a view (or any specific UI technology) exists.
- An MVVM view uses the WPF binding feature to bi-directionally associate properties exposed by controls in the view (items in a drop-down menu, or the effect of pressing a button) with the properties exposed by the view model.

Building Loosely Coupled Components (1/3)

- One of most important features of the MVC pattern is that it enables separation of concerns.
- We want the components in my applications to be as independent as possible and to have as few interdependencies as we can arrange.
- In an ideal situation, each component knows nothing about any other component and only deals with other areas of the application through abstract interfaces.
- This is known as loose coupling, and it makes testing and modifying applications easier.

Building Loosely Coupled Components (2/3)

- Example
  - If I am writing a component called MyEmailSender that will send e-mails, I would implement an interface that defines all the public functions required to send an e-mail, which I would call IEmailSender.
  - Any other component of my application that needs to send an e-mail—let's say a password reset helper called PasswordResetHelper—can then send an e-mail by referring only to the methods in the interface.
  - There is no direct dependency between PasswordResetHelper and MyEmailSender

Building Loosely Coupled Components (3/3)

- Example (cont)
  - By introducing IEmailSender, I ensure that there is no direct dependency between PasswordResetHelper and MyEmailSender.
  - I could replace MyEmailSender with another e-mail provider or even use a mock implementation for testing purposes without needing to make any changes to PasswordResetHelper.

Benefits of MVC architecture

- Greatly improved architecture
- Extensibility
- Tight Control over HTML and HTTP
- Testability
- Powerful Routing System
- Built on the Best Parts of the ASP.NET Platform
- Modern API
- ASP.NET MVC Is Open Source

Summary

- In this chapter, I introduced you to the MVC architectural pattern and compared it to some other patterns you may have seen or heard of before.
- I discussed the significance of the domain model and introduced dependency injection, which allows us to decouple components to enforce a strict separation between the parts of an application.
- I demonstrated a simple unit test and you saw how decoupled components and dependency injection make unit testing simple and easy.

Q&A

This Photo by Unknown Author is licensed under CC BY-NC-ND